Access Control Lab

Younes Menfalouti

Discretionary Access Control (DAC)

Guided Experiment

```
➜  Information-Security git:(main) cd AccessControlLab
➜  AccessControlLab git:(main) mkdir /DAC_Lab
mkdir: cannot create directory '/DAC_Lab': Permission denied
➜  AccessControlLab git:(main) mkdir DAC_Lab
➜  AccessControlLab git:(main) ls
DAC_Lab
➜  AccessControlLab git:(main) cd DAC_Lab
➜  DAC_Lab git:(main) touch report.txt
➜  DAC_Lab git:(main) ✗ ls
report.txt
➜  DAC_Lab git:(main) ✗ ls -l report.txt
-rw-r--r-- 1 phecda phecda 0 Nov 10 12:20 report.txt
➜  DAC_Lab git:(main) ✗ chmod 740 report.txt
➜  DAC_Lab git:(main) ✗ ls -l report.txt
-rwxr----- 1 phecda phecda 0 Nov 10 12:20 report.txt
➜  DAC_Lab git:(main) ✗ sudo groupadd accounting
[sudo] password for phecda:
➜  DAC_Lab git:(main) ✗ sudo chown root:accounting report.txt
➜  DAC_Lab git:(main) ✗ ls -l report.txt
-rwxr----- 1 root accounting 0 Nov 10 12:20 report.txt
```

Figure 1. DAC guided experiment

the document is set to `740` so the owner can read, modify it, and execute it, while the accounting group can only read it, and others have no access.

Challange 1

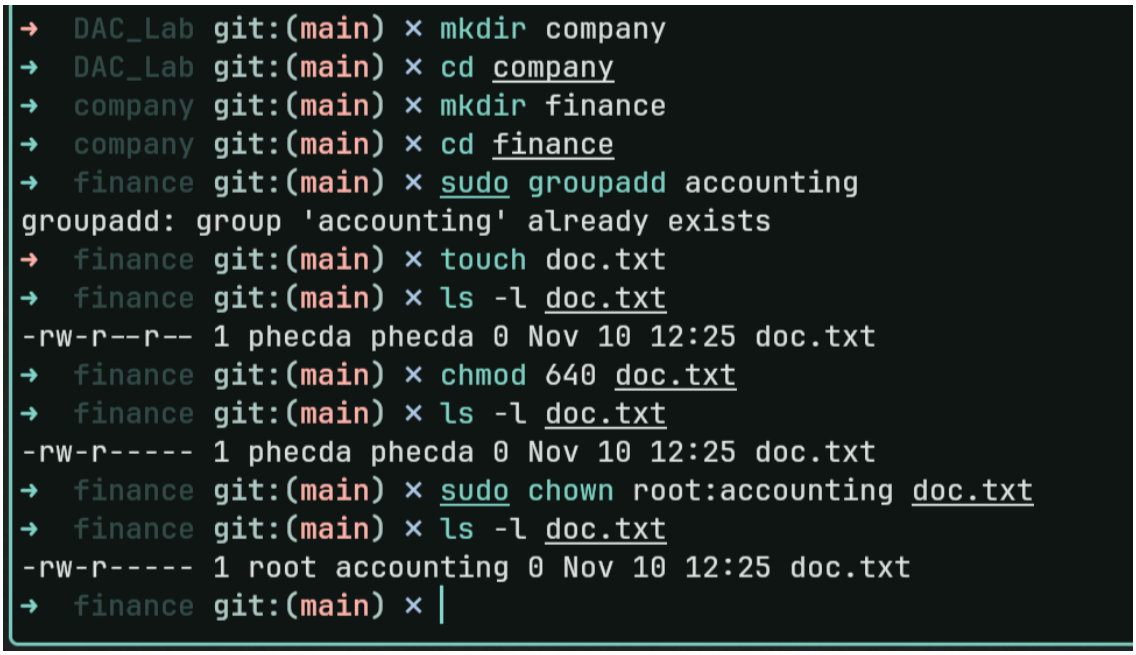


Figure 2. Caesar encryption, decryption and brute force.

Question 6. Access requires every check along the path to succeed. The directory (`750`) gives the group `r-x`, so members can enter it and list its contents. The file (`640`, i.e. `rw-r-----`), however, gives the group no permissions. Hence the group member can see the file name but reading it fails.

Mandatory Access Control (MAC)

SELinux Exploration

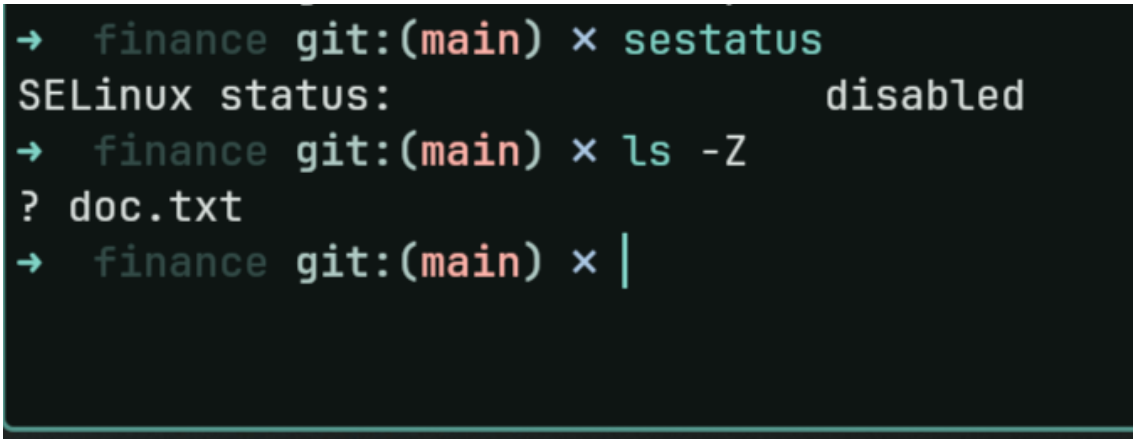


Figure 3. Output of `sestatus` and `ls -Z`.

Challenge 2

1. Access matrix (read).

	Document_A (Secret)	Document_B (Confidential)	Document_C (Public)
Alice (Secret)	yes	yes	yes
Bob (Confidential)	no	yes	yes
Charlie (Public)	no	no	yes

2. No. Document_A is Secret, above Bob's Confidential clearance (No Read Up).

3. No. Document_C is Public, below Alice's Secret level (No Write Down).

4. No Read Up stops users from reading data above their clearance, and No Write Down stops them from copying high level data into low level documents. Together, information can only flow towards higher clearance.

5. No, DAC and MAC are both enforced and access is granted only if both allow it, so the owner's denial still blocks access. The difference is that DAC is set at the owner's discretion, while MAC is a central policy based on security labels that users cannot change.

Role-Based Access Control (RBAC)

Guided Experiment

```
➜  company git:(main) ✗ sudo groupadd administrators
[sudo] password for phecda:
➜  company git:(main) ✗ sudo groupadd developers
➜  company git:(main) ✗ sudo groupadd viewers
➜  company git:(main) ✗ sudo adduser alice
sudo: adduser: command not found
➜  company git:(main) ✗ sudo useradd alice
➜  company git:(main) ✗ sudo useradd bob
➜  company git:(main) ✗ sudo useradd alice
useradd: user 'alice' already exists
➜  company git:(main) ✗ sudo useradd charlie
➜  company git:(main) ✗ sudo usermod -aG developers alice
sudo usermod -aG viewers bob
sudo usermod -aG administrators charlie
➜  company git:(main) ✗ groups alice
groups bob
groups charlie
alice : alice developers
bob : bob viewers
charlie : charlie administrators
```

Figure 4. Creating the roles and users, assigning roles and verifying membership.

`adduser` is not available on Arch Linux, so the users were created with `useradd` instead. Each user ends up in exactly one role group: alice in `developers`, bob in `viewers` and charlie in `administrators`.

```
➜  company git:(main) ✗ sudo mkdir -p source
➜  company git:(main) ✗ ls
finance  source
➜  company git:(main) ✗ sudo mkdir -p reports
➜  company git:(main) ✗ ls -l
total 12
drwxr-xr-x 2 phecda phecda 4096 Nov 10 11:25 finance
drwxr-xr-x 2 root   root   4096 Nov 10 12:32 reports
drwxr-xr-x 2 root   root   4096 Nov 10 12:32 source
```

Figure 5. Creating the organizational directories.

Challenge 3

1. The groups and users are the ones created in the guided experiment (figure above), each user being assigned to the group matching their role.

2. Following the guided experiment, `source` is owned by `root:developers` with mode `770`, and `reports` by `root:viewers` with mode `750`.

3. No. The resulting access is:

	Source code		Reports	
	Required	Obtained	Required	Obtained
Developer	read / write	read / write	read	none
Viewer	none	none	read	read
Administrator	full	none	full	none

Developers cannot read the reports, and administrators have no access to either directory, since they are neither the owning group nor covered by the `others` bits (`0`). The limitation comes from standard Unix permissions: a directory has a single owning group, so a second role cannot be given access without also opening it to everyone.

4. The missing permissions were added with ACLs, which allow extra entries for additional groups:

- `source`: `administrators:rwx`. The setgid bit (`chmod 2770`) makes new files inherit the `developers` group.
- `reports`: `developers:r-x` and `administrators:rwx`.
- Default ACLs (`setfacl -d`) were set on both directories so files created later follow the same policy.

Since the ACL was created while `source` was still `755`, the later `chmod` only updated the ACL mask, and the owning group kept `r-x`. It was corrected with `setfacl -m g::rwx` so developers can write.

```
➜  company git:(main) ✗ sudo chown root:developers source
➜  company git:(main) ✗ sudo chmod 2770 source
➜  company git:(main) ✗ sudo chown root:viewers reports
➜  company git:(main) ✗ sudo chmod 750 reports
➜  company git:(main) ✗ sudo setfacl -m g:administrators:rwx source
➜  company git:(main) ✗ sudo setfacl -m g:developers:r-x,g:administrators:rwx reports
➜  company git:(main) ✗ sudo setfacl -d -m g:developers:rwx,g:administrators:rwx source
sudo setfacl -d -m g:viewers:r-x,g:developers:r-x,g:administrators:rwx reports
➜  company git:(main) ✗ sudo setfacl -m g::rwx source
sudo setfacl -d -m g::rwx source
➜  company git:(main) ✗ ls -ld source reports
getfacl source reports
drwxrwx---+ 2 root viewers    4096 Nov 10 12:32 reports
drwxrws---+ 2 root developers 4096 Nov 10 12:32 source
```

Figure 6. Setting the base permissions and the ACL entries.

```
# file: source
# owner: root
# group: developers
# flags: -s-
user::rwx
group::rwx
group:administrators:rwx
mask::rwx
other::---
default:user::rwx
default:group::rwx
default:group:administrators:rwx
default:group:developers:rwx
default:mask::rwx
default:other::---
```

```
# file: reports
# owner: root
# group: viewers
user::rwx
group::r-x
group:administrators:rwx
group:developers:r-x
mask::rwx
other::---
default:user::rwx
default:group::r-x
default:group:administrators:rwx
default:group:developers:r-x
default:group:viewers:r-x
default:mask::rwx
default:other::---
```

Figure 7. Final ACLs of `source` and `reports`.

Others now have no access to either directory (`other::---`), and every role has exactly the rights required by the policy.

Attribute-Based Access Control (ABAC)

Challenge 4

1–2. Evaluation of the requests.

User	Department	Clearance	Time	Decision	Responsible attribute
Alice	yes	yes	yes	granted	–
Bob	no	yes	yes	denied	department (HR $\neq$ Finance)
Charlie	yes	no	yes	denied	clearance (Public $<$ Secret)
David	yes	yes	no	denied	time (22:00 $>$ 18:00)

3. Access is denied. Alice belongs to Finance, so the department condition fails, regardless of her Secret clearance or the time of the request.

4. Yes. At 17:00 the request falls within working hours, and since David's department and clearance already match, access is granted.

5. RBAC decides only on the user's role, so every new condition requires creating more roles (e.g. one per department and clearance level). ABAC combines attributes of the subject, the object, the action and the environment in a single policy, so rules like working hours or department matching are expressed directly and apply to new users and documents automatically.